VISUAL COMPUTING

ASSIGNMENT 2

Dennis Barnes

Deadline: 23:59 9.11.2026

Important note: Should be done in groups of 2-3 people

Points and report

Each exercise is graded on working code **and** on its explanation in the report. Write a report including figures for the assignment. For every exercise, explain:

- **What you did and why:** the method, the parameters you chose, and the key lines of code as short snippets (no full listings).
- **Results:** figures and tables, each with a caption that says what it shows.
- **Observations:** what you see in the results, and the answers to the questions in the exercise.

1. Perspective rectification (2 points)

Take your a photo of a flat object at a clear angle, for example a poster, a book cover or a sheet of paper.

1. Select the 4 corners of the object and rectify the object to a frontal view with `getPerspectiveTransform` and `warpPerspective`.
2. Now try the same with an affine transformation: compute `getAffineTransform` from 3 of the 4 corners and warp the photo with it. Mark where the 4th corner ends up and report its distance in pixels to the real corner.
3. Explain why an affine transformation cannot do this. How can you see in your photo that it shows a perspective and not an affine transformation?

2. Gaussian and Laplacian pyramid (2 points)

A. Gaussian pyramid (1 point). Implement a Gaussian pyramid with 4 levels below the original image. Blur with the sampled Gauss filter [1 4 6 4 1]/16 in x and in y, then keep every second row and column. Show all levels. What are the low frequencies, and which details disappear first?

B. Laplacian pyramid (1 point). Build and show the Laplacian pyramid from your Gaussian pyramid. Then reconstruct the original image from the Laplacian levels and the smallest Gaussian level, and report the largest difference to the original. What does each Laplacian level contain, and what could the pyramid be used for?

3. Gabor filter bank (2 points)

A Gabor filter responds to stripes and edges of one orientation and one frequency.

1. Create Gabor kernels for 4 orientations (0°, 45°, 90°, 135°) with `getGaborKernel`. Filter one of your images with each kernel (`filter2D` with a float output) and combine the four results into one image by taking the maximum value of each pixel. Show each kernel next to its result.
2. Change one parameter at a time and explain what each parameter does to the kernel and to the result. Which setting finds the structures in your image best? Compare the combined result with the Sobel gradient magnitude from the PS: what does the Gabor filter bank find that Sobel does not, and the other way round?

4. Canny with automatic thresholds (2 points)

1. Set the two Canny thresholds automatically with Otsu's method.
2. Compare the result using two images with the edge detectors from the PS: Sobel, Laplacian, and Canny with hand-tuned thresholds. Rank the methods from worst to best and say which criteria you used for the ranking.

5. Feature detectors: benchmark and homography (4 points)

The number of matches alone says little, because many matches can be wrong. Build a small benchmark that checks every match, and then use the best detector on real photos.

A. Benchmark (2 points). Write a program that:

1. Takes one image and creates a transformed copie using `warpPerspective`: rotation, scaling and a perspective tilt.
2. Detects keypoints and computes descriptors with ORB, AKAZE and BRISK.
3. Count a match as correct if the keypoint from the first image, transformed with `perspectiveTransform`, lands within 3 pixels of its matched keypoint.
4. Record for each detector and transformation: kept matches, correct matches, precision (correct / kept) and runtime.

B. Experiments (1 point). Plot the number of correct matches against rotation angle and against the scale factor (4 different values each).

- Which detectors are rotation invariant, and which are scale invariant?
- Is the ranking the same for every type of transformation?

C. Homography from real photos (1 point). Take two photos of the same flat scene from different positions, for example a book on a table, or a wall with posters. Match them with the detector you chose, estimate the transformation H with `findHomography` and RANSAC, and warp one photo onto the other. Show the result and report the number of RANSAC inliers. Why do you need RANSAC here, but not in Exercise 1?

Hint: RANSAC repeatedly picks 4 random matches, computes H from them and counts how many matches agree with this H, i.e. land within a few pixels of their matched keypoint (the inliers). The H with the most inliers wins. `findHomography(pts1, pts2, RANSAC, 3.0, mask)` *does all of this; 3.0 is the distance in pixels and* `mask` *marks the inliers.*

Hint: ORB is in the main OpenCV library. AKAZE and BRISK are in the main library in OpenCV 4; in OpenCV 5 they moved to opencv_contrib (`cv::xfeatures2d`*).*

Deliverable: one zip file per group with

- the report as a PDF, with the names of all group members on the first page;
- your source code (`.cpp`, `.h` and `CMakeLists.txt`)